Poster Master Reference — v3 & v4

Bu dosyayı sindirdikten sonra posterdeki **her sayıyı ve her cümleyi** açıklayabiliyor olmalısınız. Bölüm 1–6 Türkçe (terimler İngilizce), Bölüm 7 (Q&A) tamamen İngilizce.

---

1. Projenin özü (3 cümlede)

Black–Scholes modelinde European call/put fiyatını Monte Carlo ile hesaplıyoruz; bu parametrelerde closed-form çözüm var (10.4506), dolayısıyla her sonucu **kesin doğru cevapla** karşılaştırabiliyoruz. Tek çekirdekli CPU baseline'a karşı üç CUDA kernel'i yazdık (naive → shared-memory reduction → antithetic variates); her biri **tek bir performans dersini** izole ediyor. Hepsine uygulanan grid-stride refactor ile birlikte sonuç: 10^7 path'te **382× end-to-end** speedup, kernel-only **≈1652×**, tüm sonuçlar analitik fiyatla istatistiksel olarak tutarlı.

---

2. Finans temeli

Option / call / put. Call option: vade T'de, sabit strike fiyatı K'dan hisseyi **alma hakkı** (zorunluluk değil). Vadede hisse S_T ise kazanç (payoff): $g = \max(S_T - K, 0)$. Put aynısının satma hakkı: $\max(K - S_T, 0)$. Fiyatlama sorusu: bu hakkın bugünkü adil fiyatı nedir?

Geometric Brownian Motion (GBM). Black–Scholes varsayımı: hisse fiyatı $dS = r \cdot S \cdot dt + \sigma \cdot S \cdot dW$ izler (risk-neutral). Bu SDE'nin **kapalı çözümü** var:

$$S_T = S_0 \cdot \exp((r - \sigma^2/2) \cdot T + \sigma \cdot \sqrt{T} \cdot Z), \quad Z \sim N(0,1)$$

Bu yüzden European option için zaman adımlamaya gerek yok — **tek adım exact** (discretization error sıfır). Kernel'lerde thread başına yapılan işlem tam olarak bu tek satır + payoff.

Parametreler ve ground truth. S_0=100, K=100, r=0.05, σ=0.2, T=1. Black–Scholes formülü:

$$C = S_0 \cdot N(d_1) - K \cdot e^{(-rT)} \cdot N(d_2)$$
$$d_1 = [\ln(S_0/K) + (r + \sigma^2/2)T] / (\sigma\sqrt{T}) = 0.35 \qquad N(d_1) \approx 0.6368$$
$$d_2 = d_1 - \sigma\sqrt{T} = 0.15 \qquad N(d_2) \approx 0.5596$$
$$C = 63.68 - 95.12 \cdot 0.5596 \approx 10.4506$$

Put fiyatı put–call parity'den: $P = C - S_0 + K \cdot e^{(-rT)} \approx 5.5735$.

Monte Carlo estimator. $\hat{C} = e^{(-rT)} \cdot (1/N) \cdot \Sigma\, g(Z_i)$. Central Limit Theorem gereği hata $SE = s/\sqrt{N}$ ile küçülür (s = payoff'ların sample standard deviation'ı, bizim parametrelerde $s \approx 14.7$). Bu yüzden:

- 10× daha iyi doğruluk = 100× daha fazla path → hesap pahalı → GPU gerekçesi.
- $N=10^6$'da $SE \approx 14.7/1000 \approx$ **0.0147** → validation tablosundaki σ kolonu = $|\hat{C} - 10.4506| / SE$.
- Host tarafında kernel'in döndürdüğü Σg ve Σg^2den s, SE ve %95 CI ($\pm 1.96 \cdot SE$) hesaplanır.

Antithetic variates teorisi. Z ile −Z'nin payoff'ları negatif korelasyonlu (payoff Z'de monoton olduğu için). Çift ortalamasının varyansı:

$$\mathrm{Var}(\tfrac{1}{2}(g^+ + g^-)) = \tfrac{1}{2}\cdot\mathrm{Var}(g)\cdot(1 + \rho), \quad \rho = \mathrm{corr}(g^+, g^-) < 0$$

ρ negatif olduğu için varyans yarıdan da aza inebilir; posterde muhafazakâr olarak "~½ variance" diyoruz. Tabloda doğrulaması: antithetic σ=0.8 vs shared σ=1.4 → varyans oranı $(0.8/1.4)^2 \approx 0.5$ ✓Maliyeti: pair başına bir ekstra exp(); üstelik RNG state pair başına **bir kez** okunur.

---

3. CUDA tarafı

mc_naive — one thread, one path. Her thread kendi curandState'inden Z çeker, S_T ve payoff'u hesaplar, sonucu atomicAdd ile global d_sum ve d_sum^2'ye yazar (path başına **2 atomic** → toplam ~2·N). Milyonlarca thread **aynı iki adrese** yazınca atomic'ler L2'de serialize olur. Nsight: occupancy %94 ama SM (compute) throughput %2.8 — çekirdekler zamanın ~%97'sinde bekliyor. Ders: **occupancy ≠ utilization**.

mc_shared — hierarchical reduction. Simülasyon aynı; toplama akıllı: (1) thread kendi register'ında biriktirir, (2) warp içinde __shfl_down_sync ile 5 adımda (offset 16,8,4,2,1) toplanır, (3) her warp'ın lane-0'ı shared memory'ye yazar (256 thread = 8 warp), (4) blok başına **tek** atomicAdd. Grid 64 blok × 2 accumulator (sum, sum^2) = **~128 atomic** — N'den bağımsız. SM throughput %2.8 → **%79**.

mc_antithetic — ±Z pairs. mc_shared'in üstüne: her state okumasından Z çek, hem +Z hem −Z payoff'unu hesapla, ortalamasını biriktir. Effective sample başına maliyet mc_shared'den düşük (RNG + Box–Muller maliyeti ikiye bölünüyor), varyans yarı.

Grid-stride refactor (hepsine uygulandı). Önce: thread sayısı = N → curandState dizisi 48 B × N ($N=10^6$'da ~48 MB) ve init_rng çalışma süresini domine ediyor (44.6 ms). Sonra: **sabit 16,384 thread** (64 blok × 256 thread); her thread for (i = tid; i < N; i += 16384) döngüsüyle çok path işler, state'ini **bir kez** okur, register'da biriktirir. Init 0.68 ms (**65×**), state bellek trafiği O(N)→O(threads) → kernel'ler **memory-bound'dan compute-bound'a** döner. v4'teki roofline paneli bunun görsel kanıtı.

cuRAND detayları. Generator: **XORWOW** (state 48 byte). Normal sayılar curand_normal → uniform + **Box–Muller**. Bağımsızlık: curand_init(seed, tid, 0, &state) — her thread'e ayrı **subsequence**, aynı seed. Asla iki thread aynı state'i paylaşmaz.

---

4. Ölçüm metodolojisi

- **Sweep:** N ∈ {10^4 … 5×10^7}, her boyutta 5 run (seed 1–5), **median** raporlanır (outlier'a dayanıklı).
- **Warm-up:** her batch'ten önce bir ısınma launch'ı — ilk çağrıdaki JIT/module-load ve soğuk cache etkisi atılır.
- **Zamanlama:** GPU = CUDA events (cudaEventRecord, GPU saatinde); CPU = wall-clock (Windows QPC / clock_gettime).
- **End-to-end tanımı:** RNG init + kernel + sonucun host'a okunması. Bar chart başlığındaki "(RNG init + kernel)" bu demek; cherry-picked kernel süresi değil.
- **Validation gate:** her run analitik fiyata 4σ içinde olmalı (otomatik kabul eşiği; yanlış alarm olasılığı ~6×10^{-5}). Tablodaki 0.8–1.4σ değerleri bu eşiğin çok altında.
- **Donanım:** NVIDIA GTX 1650 **laptop** (TU117, sm_75, **16 SM = 1024 core**, 4 GB GDDR5, 128 GB/s) — masaüstü 1650'nin 14 SM'inden farklıdır, sorulursa: laptop varyantı. FP32 peak = 2 FLOP (FMA) × 1024 core × sürekli gözlenen ~1.22 GHz ≈ **2490 GFLOPS** (roofline'daki ceiling). CPU: AMD Ryzen 7 5800H (8C/16T), tek thread ~**30 M paths/s**. CUDA 12.4, host MSVC, FP32.

---

5. Posterdeki her sayı

Önce: beş kilit sayı (ezberlenecekler)

Bu beş sayı posterin tüm hikayesi: ikisi *ne kadar hızlı* (382×/980×, 1652×), ikisi *neden hızlı* (2.8→79%, 65×), biri *doğru mu* (0.8σ).

382× / 980× — end-to-end speedup. Aynı iş (N path simüle edip fiyat üretmek) iki tarafta da baştan sona ölçülüyor. CPU tek thread ~30 M path/s → 10^7 path ≈ 360 ms; GPU'da en iyi kernel (mc_antithetic, RNG init + kernel + readback dahil) ≈ 0.9 ms → **382×**. 5×10^7'de: ≈1.6 s vs ≈1.6 ms → **980×**. İki sayı vermemizin sebebi dürüstlük: N büyüdükçe GPU'nun sabit maliyetleri amortize oluyor, speedup büyüyor.

≈1652× — kernel-only speedup. Aynı karşılaştırma ama sadece simülasyon kernel'i (init hariç). 980× ile birlikte okunur: "tek fiyat soğuk başlangıçla 980×; arka arkaya çok fiyat hesaplarken (init bir kez ödenir) 1652×'e yaklaşılır." v3'te ≈1650 diye yuvarlanmış; ölçülen tam değer 1652.

2.8% → 79% — SM throughput (posterin asıl dersi). Nsight metriği: çekirdekler teorik tepenin yüzde kaçında gerçek iş yaptı. Naive'de her path aynı iki global adrese atomicAdd yapıyor; aynı adrese atomic'ler L2'de sıraya girer → çekirdekler ~%97 boşta → %2.8 (occupancy %94 olmasına rağmen — *resident ≠ working*). Shared'de toplama hiyerarşik:

register → warp shuffle → shared memory → blok başına tek atomic (~128 toplam) → %79. **Matematik aynı, sadece toplama yöntemi değişti**, kullanım 28 kat arttı.

65× — grid-stride'ın init kazancı. Path başına bir thread → N tane cuRAND state (48 B × 10^6 ≈ 46 MB) → init_rng 44.6 ms (kernel'den uzun!). Sabit 16.384 thread → 16.384 state → 0.68 ms = **65×**. Asıl önemli yan etki: state trafiği kalkınca kernel'ler memory-bound'dan **compute-bound**'a döner (v4 roofline panelinin gösterdiği şey).

0.8σ — doğruluk kanıtı. σ birimi = Monte Carlo standard error (SE = s/√N). Antithetic N=10^6'da: tahmin 10.4593, analitik 10.4506 → hata 0.0088; kendi SE'si ≈ 0.011 → **0.8σ** — sapma tamamen örnekleme gürültüsü, bug yok. ~1σ civarı değerler unbiased estimator için *sağlıklıdır*; 4σ kabul eşiğiyle karıştırmayın.

Sunum cümlesi (bu sırayla): "382× hızlıyız end-to-end — kernel-only 1652× — çünkü atomic darboğazını çözdük (2.8% → 79%) ve RNG init'i 65× ucuzlattık; sonuç da doğru: analitik fiyata 0.8σ mesafede."

Tüm sayılar (tek tek)

Sayı
10.4506
1/√N
2.8% (2.75%)
94%
~97% idle
~2·N → ~128
79% / 76%
½ variance
$\hat{g}$ = ½[g(+Z)+g(−Z)]
48 B × N → × 16,384
16,384
44.6 → 0.68 ms (65×)
1.06 → 0.08 ms (13×)
0.61 → 0.07 ms (8.7×)
10× / 13×
348× / 772×
382× / 980×
≈1650× (1652×)
≈30 M/s
≈415 M/s
"tens of G paths/s"
1.6 s vs ~1 ms

10.4501 / 0.0005 / 0.0σ
10.4718 / 0.0212 / 1.4σ
10.4719 / 0.0213 / 1.4σ
10.4593 / 0.0088 / 0.8σ
4σ
2490 GFLOPS
128 GB/s
sm_75 / CUDA 12.4 / -O3 -lineinfo
200×
seeds 1–5, N ∈ {10^4…5×10^7}

6. v3 ↔ v4 farkları

Konu
Overview
Roofline paneli
Alt şerit (hero band)
σ iddiası
Validation caption
Throughput caption
Kernel-only sayısı

v3 sunan kişi Bölüm 7'deki D1 ve D2 sorularının cevaplarını ezbere bilmeli (1σ çelişkisi ve naive eğrisinin yükselmesi).

---

7. Q&A — fully English

A. Finance & math

A1. What exactly is a European call option? The right, but not the obligation, to buy the underlying stock at a fixed strike price K at expiry T. If the stock ends at S_T, the holder earns max(S_T − K, 0). "European" means it can only be exercised at expiry — unlike American options, which allow early exercise.

A2. Monte Carlo is about simulating paths — why does each thread do only one step? Under GBM the SDE has an exact lognormal solution, so for a payoff that depends only on the terminal price we can jump straight to S_T in one step — zero discretization error. Path-dependent options (e.g., Asian) would need a time-stepping loop; our design point "one thread = one path" was chosen with that extension in mind.

A3. Where does 10.4506 come from? The Black–Scholes closed form with $S_0=K=100$, $r=0.05$, $\sigma=0.2$, $T=1$: $d_1=0.35$, $d_2=0.15$, $C = 100\cdot\Phi(0.35) - 100\cdot e^{(-0.05)}\cdot\Phi(0.15) \approx 10.4506$. We treat it as an exact oracle for validation.

A4. If a closed form exists, why use Monte Carlo at all? Precisely because the answer is known, it is the perfect testbed: every kernel can be validated to statistical precision. The engineering — RNG management, reductions, variance reduction — transfers unchanged to payoffs with no closed form, which is where MC is actually used in industry.

A5. How do antithetic variates work, and when do they fail? For each draw Z we also evaluate −Z and average the two payoffs. Because the call payoff is monotone in Z, the pair is negatively correlated and $\mathrm{Var}(\tfrac{1}{2}(g^+ + g^-)) = \tfrac{1}{2}\mathrm{Var}(g)(1+\rho) < \tfrac{1}{2}\mathrm{Var}(g)$. It can fail (no benefit) for payoffs that are not monotone in Z — e.g., some barrier or straddle-like payoffs where $g(Z)$ and $g(-Z)$ are positively correlated.

A6. What does the σ column in the validation table mean? The absolute error divided by the Monte Carlo standard error $SE = s/\sqrt{N}$ ($s \approx 14.7$, so $SE \approx 0.0147$ at $N=10^6$). It answers: "is the deviation explainable by sampling noise?" Values around 1 are exactly what an unbiased estimator should produce; consistently getting 0 would itself be suspicious.

A7. Why does the CPU row show 0.0σ? Luck of the seed, plus rounding: its error 0.0005 is ≈0.03·SE, which rounds to 0.0. It does not mean the CPU is "more correct" — all four estimators have the same statistical quality per path.

A8. Why are the naive and shared estimates almost identical (10.4718 vs 10.4719)? They consume the same cuRAND sequences with the same seeds, so they see the same random draws; they differ only in summation order, which changes the result by float rounding only. Antithetic differs because its sampling scheme (±Z pairs) is genuinely different.

B. CUDA & performance

B1. Occupancy is 94% but compute throughput is 2.8% — how can both be true? Occupancy counts how many threads are *resident*; throughput measures whether they *do work*. In mc_naive nearly all threads are parked waiting for their atomicAdd to the same two global addresses; atomics to one address serialize in L2. Resident-but-stalled is the textbook trap: occupancy is not utilization.

B2. Is the naive kernel memory-bandwidth-bound then? No — memory throughput is only ~8% of peak. It is bound by *atomic contention*: a serialization bottleneck, not a bandwidth one. That is visible in the throughput plot as a hard ceiling at ≈415 M paths/s regardless of N.

B3. Where does "~128 atomics" come from? The fixed grid has 64 blocks; after the block-level reduction each block issues one atomicAdd per accumulator, and we keep two accumulators (Σg and Σg^2, needed for the confidence interval). 64 × 2 = 128, independent of N.

B4. Why reduce with warp shuffles before shared memory? __shfl_down_sync moves data register-to-register within a warp — no shared-memory traffic, no bank conflicts, no __syncthreads(). Five shuffle steps (offsets 16, 8, 4, 2, 1) collapse 32 lanes to one; only the 8 warp leaders touch shared memory. It is the standard modern reduction pattern.

B5. The optimized kernels have *lower* occupancy (≈73–75%) than naive (94%) yet run far faster. Why? They use more registers per thread (private accumulators), which limits resident threads. But each thread does vastly more useful work per cycle. We optimized for throughput, not occupancy — chasing occupancy for its own sake is exactly the mistake the naive kernel demonstrates.

B6. What is a grid-stride loop and why 16,384 threads? Instead of one thread per path, a fixed grid where thread i processes paths i, i+16384, i+2·16384, … 16,384 = 64 blocks × 256 threads — enough to saturate all 16 SMs with multiple warps each, small enough that RNG state stays tiny. 256 threads/block is a sweet spot for the reduction (8 warps) and register pressure; nearby sizes performed within noise.

B7. Why was RNG initialization so expensive before the refactor? curand_init for XORWOW does sequence skip-ahead work per state, and one-thread-per-path needs N states — 48 B × 10^6 ≈ 46 MB of state written at N=10^6 (44.6 ms, longer than the kernel itself). With 16,384 states it drops to 0.68 ms (65×) and is amortized across any N.

B8. Why XORWOW? Did you consider Philox or MRG32k3a? XORWOW is cuRAND's default device generator: smallest state, fastest init, statistically adequate for pricing (it passes standard test batteries for this use). Philox is counter-based — stateless and skip-friendly, an attractive alternative we'd try first if we extended the study; MRG32k3a has stronger guarantees but is slower.

B9. Why FP32, and what about the precision breakdown at 5×10^7? Consumer GPUs run FP64 at ~1/32 the FP32 rate, and pricing accuracy is dominated by statistical error (SE ⩾ rounding) at practical N. The validation plot makes the limit visible: naive's single FP32 accumulator degrades at 5×10^7 because tiny payoffs are added to a huge running sum. The hierarchical reduction largely avoids this by summing in registers first (pairwise-like partial sums). Remedies if needed: double or Kahan accumulators just for the reduction — a few % cost.

B10. Why is the antithetic kernel cheaper per effective sample than mc_shared? One state read and one Box–Muller draw yield *two* payoff evaluations, whose average is one effective sample. The RNG cost — the dominant per-sample cost in the compute-bound regime — is halved, at the price of one extra exp(). Measured: 0.07 ms vs 0.08 ms per 10^6 effective samples.

B11. Why not use CUB / thrust device-wide reduction? We wanted the reduction inside the same kernel that generates payoffs (no intermediate N-sized array — that array alone would be 4 MB at 10^6 and would re-introduce memory traffic). Writing it manually is also the point of the course: the warp-shuffle pattern is what CUB implements internally anyway.

B12. How do you know the reduction is race-free? Within a warp, __shfl_down_sync with a full mask is synchronous by construction; across warps we use shared memory with __syncthreads() between write and read; across blocks the only shared state is the two atomic accumulators. Validation against the analytical price at every N is the end-to-end regression test.

C. Methodology & validation

C1. Is 382× a fair number? You compare against a single CPU thread. We state the baseline explicitly: one core of a Ryzen 7 5800H, -O3, and we time the GPU end-to-end (RNG init + kernel + readback), median of 5. A perfectly scaled 16-thread CPU version would shrink the gap to roughly 25–60×, which we acknowledge — but per-core comparison is the standard way to isolate architectural speedup, and the kernel-vs-kernel lessons (2.8% → 79%) are baseline-independent.

C2. Why median of 5 runs, and why discard a warm-up? The first launch pays one-time costs (driver/JIT, cold caches) that are not steady-state performance; the median of the remaining runs is robust to OS scheduling outliers. Run-to-run spread was small (a few %), so 5 runs suffice.

C3. How exactly do you time CPU and GPU? GPU with CUDA events recorded on the stream around the region of interest (device-side clock, no host jitter); CPU with QueryPerformanceCounter / clock_gettime. We never compare a GPU event time to a CPU wall time within one number — end-to-end GPU numbers are wall-clock too.

C4. What is the 4σ acceptance gate? Why 4? Every benchmark run's price must lie within 4 standard errors of the analytical value, or the run is rejected as broken. Under a correct implementation that false-alarm probability is $\sim 6\times10^{-5}$, so the gate catches real bugs (wrong RNG usage, race conditions) without ever tripping on honest noise. The 0.8–1.4σ values in the table show typical healthy runs.

C5. Why benchmark on a laptop GTX 1650 instead of Colab's T4? Stable, exclusive access and fixed clocks — Colab VMs share hardware and throttle unpredictably, which ruins medians. The 1650 (laptop, TU117, 16 SM) is the same Turing architecture (sm_75) as the T4, so the lessons transfer; absolute speedups would only grow on a bigger GPU.

C6. How close are you to the hardware's peak? Nsight reports the optimized kernels at 76–79% of peak SM throughput; the roofline (v4) shows both sitting essentially on the FP32 compute ceiling (≈2.49 TFLOPS at observed clocks). The remaining ~20% is special-function (exp/log) pipe pressure and reduction overhead — there is no low-hanging fruit left in this kernel.

D. Hard / skeptical questions

D1. (v3) Your conclusions claim "within 1σ" but your own table says 1.4σ. Correct — the precise statement is that every kernel is statistically *consistent* with the analytical price: ≤1.4σ at $N=10^6$, antithetic at 0.8σ, all far inside our 4σ gate. The v4 poster words it exactly that way. (Deviations near 1σ are expected behavior for an unbiased estimator — observing many runs all at $\ll 1\sigma$ would actually indicate a bug or inflated error bars.)

D2. The naive error curve *rises* at 5×10^7 — is your simulation wrong? No — that is FP32 accumulation breakdown, and we keep it on the poster deliberately. Adding $\sim10^{-1}$-scale payoffs into a sum of magnitude $\sim5\times10^8$ loses them to rounding; atomicAdd order makes it worse. The hierarchically-reduced kernels don't show it. It turns the plot into a correctness argument for the optimized reduction, not just a performance one.

D3. This problem is embarrassingly parallel — what is the actual contribution? That parallelism is *available* doesn't mean it's *achieved*: the naive kernel is also "embarrassingly parallel" and reaches 2.8% of peak — a 38× gap to the optimized version at identical arithmetic. The contribution is the profiler-driven diagnosis of each bottleneck (atomics → reduction; RNG state → grid-stride) with each fix justified by a measured Nsight signal, plus the validated end-to-end methodology.

D4. Why no real market data? Correctness doesn't need it: the analytical price is a stronger oracle than any market quote, which embeds spreads, dividends, and implied-vol structure. Market data is a demo garnish (we considered yfinance for a live example), not a validation tool — and the course grade is on parallel performance engineering.

D5. Your speedup grows with N (382× → 980×). Isn't that suspicious? It's expected: GPU fixed costs (init, launch) amortize as N grows while the CPU scales linearly, and the GPU only saturates all SMs once N is large. That's why we report the whole sweep 10^4… 5×10^7 rather than one flattering point — at small N the GPU advantage is genuinely smaller.

E. Extensions / future work

E1. How would you price Asian or American options? Asian (path-dependent): add a time-stepping loop per thread — the "one thread = one path" mapping and the entire reduction machinery stay; arithmetic intensity rises, which favors the GPU even more. American: needs early-exercise decisions, e.g., Longstaff–Schwartz regression — a different algorithm with per-step global reductions; substantially harder, good future work.

E2. Multi-GPU or CUDA streams? Single GPU saturates at these problem sizes, so streams mainly help overlap init/transfer with compute — that was our "hope to achieve" item (timeline overlap in Nsight Systems). Multi-GPU is near-linear for MC: split paths across devices, reduce partial sums at the end.

E3. What would FP64 cost? On this consumer GPU, FP64 peak is 1/32 of FP32, so a naive switch would be catastrophic; the sensible design is FP32 sampling with FP64 (or Kahan) accumulators in the reduction — accuracy where it matters at negligible cost. We'd add it as a build flag (-DUSE_F64_ACC).

E4. Could you use the tensor cores / TF32? Not naturally — this workload is scalar special-function math (exp, log, sincos via Box–Muller), not matrix multiply. The win would come from batching many *options* (different strikes/maturities) per path, which changes the problem shape.